

CSU33032 Project 2 – Securing the Cloud

Lucia Brown – 22336688

My submission consists of two Python files – one containing the secure cloud functionality (secure.py) and a test script with three test cases. The test cases ensure that specified users are able to decrypt files available on the server; that newly added users can also access the functionality; and that removed users can no longer decrypt.

Secure.py Overview:

generate_user_cert: This function generates public and private keys for the users using RSA with a key size of 2048. The generated files are in the format {username}_public.pem and {username}_private.pem.

load_public_key_from_cert: This function extracts the user's public key from their generated public.pem file

encrypt_aes_key: This function encrypts the AES key, using a public RSA key.

decrypt_aes_key: This function decrypts the AES key, using a private RSA key.

encrypt_file: This function generates an AES-256 key, reads in plaintext and uses the AES key to encrypt the plaintext. The encrypted ciphertext is written as a document in the format {filepath}.enc

upload_to_cloud: This function uploads the sample file to both GoogleDrive and Dropbox in parallel. The sample.txt.key and sample.txt.enc are uploaded.

decrypt_file: This function takes in the path of the encoded file, a username and the user's private key path. It looks for the encrypted file at the desired file path, checks if the user can decrypt the file and if they can, their private key is used for decryption.

add_user_to_group: This function takes in a username and a person's public key as parameters and adds the person to the user group of those who are able to decrypt the files from the cloud. It loads the public key and encrypts the AES key for the

remove_user_from_group: This function takes in a username and searches for that username in the user group. If the user is found, they and their key will be removed from the user group and will lose access to decryption.

Test Script.py Overview:

There are three test cases in this script.

Test 1: This test adds Alice and Bob to the user group and ensures that they are able to decrypt the file while securely stored on the cloud.

Test 2: After the user group is initialised with just Alice and Bob, Charlie is added to the user group. This test ensures that Charlie is also able to decrypt and access the file.

Test 3: Bob is removed from the user group and should no longer be able to decrypt the file.

Secure.py:

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import serialization, hashes
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography import x509
from cryptography.x509.oid import NameOID
from datetime import datetime, timedelta, timezone
import json, os, boto3, base64, threading, dropbox
from dotenv import load_dotenv
from pydrive.auth import GoogleAuth
from pydrive.drive import GoogleDrive

load_dotenv()
dbx = dropbox.Dropbox(os.getenv("DROPBOX_ACCESS_TOKEN"))
aws_region = "us-east-1"
kms_key_id = os.getenv("KMS_KEY_ID")
kms_client = boto3.client("kms", region_name=aws_region)

#Create a certificate for the user
def generate_user_cert(username):
    """ Generate and store user certificates. """
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )
    subject = issuer = x509.Name([
        x509.NameAttribute(NameOID.COUNTRY_NAME, "US"),
        x509.NameAttribute(NameOID.ORGANIZATION_NAME, "SecureCloudGroup"),
        x509.NameAttribute(NameOID.COMMON_NAME, username),
    ])
    cert = (
        x509.CertificateBuilder()
        .subject_name(subject)
        .issuer_name(issuer)
```

```

        .public_key(private_key.public_key())
        .serial_number(x509.random_serial_number())
        .not_valid_before(datetime.now(timezone.utc))
        .not_valid_after(datetime.now(timezone.utc) + timedelta(days=365))
        .sign(private_key, hashes.SHA256())
    )
    with open(f"{username}_public.pem", "wb") as f:
        f.write(cert.public_bytes(serialization.Encoding.PEM))
    # Store private key
    with open(f"{username}_private.pem", "wb") as f:
        f.write(private_key.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.PKCS8,
            encryption_algorithm=serialization.NoEncryption()
        ))
    return private_key

#Extract the public key from the generated _public.pem certificate
def load_public_key_from_cert(cert_path):
    """Load a public key from a PEM-encoded certificate."""
    try:
        with open(cert_path, "rb") as f:
            cert = x509.load_pem_x509_certificate(f.read())
            public_key = cert.public_key()
            # Ensure it's an RSAPublicKey object
            if isinstance(public_key, rsa.RSAPublicKey):
                return public_key
            else:
                print(f"Error: Public key in certificate is not of type
RSA PublicKey.")
                return None
    except Exception as e:
        print(f"Error loading public key from certificate: {e}")
        return None

def encrypt_aes_key(aes_key, public_key):
    """Encrypt the AES key with an RSA public key."""
    try:
        # Encrypt AES key using RSA public key
        encrypted_aes_key = public_key.encrypt(
            aes_key,
            padding.OAEP(
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )
    
```

```

    )
    )
    # Return the encrypted AES key
    return base64.b64encode(encrypted_aes_key).decode()
except Exception as e:
    print(f"Error encrypting AES key: {e}")
    return None

def decrypt_aes_key(encrypted_aes_key, private_key_path):
    """Decrypt the AES key using an RSA private key with `cryptography`."""
    try:
        # Load the RSA private key
        with open(private_key_path, "rb") as f:
            private_key = serialization.load_pem_private_key(f.read(),
password=None)
        # Decrypt the AES key
        decrypted_aes_key = private_key.decrypt(
            base64.b64decode(encrypted_aes_key),
            padding.OAEP(
                mgf=padding.MGF1(algorithm=hashes.SHA256()),
                algorithm=hashes.SHA256(),
                label=None
            )
        )
        return decrypted_aes_key
    except Exception as e:
        print(f"Error decrypting AES key: {e}")
        return None

def encrypt_file(file_path, user_keys):
    """ Encrypt a file using AES and encrypt the key for each user. """
    aes_key = os.urandom(32) # Generate a secure 256-bit AES key
    iv = os.urandom(16)
    cipher = Cipher(algorithms.AES(aes_key), modes.GCM(iv))
    encryptor = cipher.encryptor()

    try:
        with open(file_path, "rb") as f:
            plaintext = f.read()
    except FileNotFoundError:
        print(f"Error: File {file_path} not found.")
    return

```

```

ciphertext = encryptor.update(plaintext) + encryptor.finalize()
tag = encryptor.tag

enc_file_path = file_path + ".enc"
with open(enc_file_path, "wb") as f:
    f.write(iv + tag + ciphertext)

#Load public keys from certificates
loaded_keys = {}
for user, cert_path in user_keys.items():
    loaded_keys[user] = load_public_key_from_cert(cert_path)
    if loaded_keys[user] is None:
        print(f"Skipping encryption for {user}: Could not load public key.")

enc_keys = {user: encrypt_aes_key(aes_key, pub_key) for user, pub_key in
loaded_keys.items() if pub_key}

key_file_path = file_path + ".key"
with open(key_file_path, "w") as f:
    json.dump(enc_keys, f)

return enc_file_path, key_file_path

def upload_to_cloud(file_path):
    """ Upload file to both Dropbox and Google Drive in parallel """
    def upload_to_dropbox():
        with open(file_path, "rb") as f:
            dbx.files_upload(f.read(),
f"/{file_path}",mode=dropbox.files.WriteMode.overwrite, mute=True)
        print(f"Dropbox: {file_path} uploaded!")

    def upload_to_google_drive():
        gauth = GoogleAuth()
        gauth.LocalWebserverAuth()
        drive = GoogleDrive(gauth)
        file_drive = drive.CreateFile({'title': os.path.basename(file_path)})
        file_drive.SetContentFile(file_path)
        file_drive.Upload()
        print(f"Google Drive: {file_path} uploaded!")

    # Run both uploads in parallel
    dropbox_thread = threading.Thread(target=upload_to_dropbox)
    drive_thread = threading.Thread(target=upload_to_google_drive)

```

```

dropbox_thread.start()
drive_thread.start()
dropbox_thread.join()
drive_thread.join()

if os.path.exists("sample.txt.key"):
    upload_to_cloud("sample.txt.key")
else:
    print("Error: sample.txt.key not found. Ensure encryption was successful.")

upload_to_cloud("sample.txt.enc")
upload_to_cloud("sample.txt.key")

def decrypt_file(enc_file_path, private_key_path, username):
    """Decrypts an encrypted file using the provided private key."""
    key_file_path = enc_file_path.replace(".enc", ".key")
    print(f"Looking for key file: {key_file_path}")
    try:
        with open(key_file_path, "r") as f:
            enc_keys = json.load(f)
    except FileNotFoundError:
        print(f"Error: The key file {key_file_path} was not found.")
        return
    except json.JSONDecodeError as e:
        print(f"Error reading the key file: {e}")
        return
    if username not in enc_keys:
        print(f"Decryption failed: No AES key found for user {username}.")
        return
    # Load private key
    aes_key = decrypt_aes_key(enc_keys[username], private_key_path)
    if aes_key is None:
        print("Decryption failed: Could not retrieve AES key.")
        return
    try:
        with open(enc_file_path, "rb") as f:
            iv, tag, ciphertext = f.read(16), f.read(16), f.read()

            cipher = Cipher(algorithms.AES(aes_key), modes.GCM(iv))
            decryptor = cipher.decryptor()
            decryptor.authenticate_additional_data(b"")
            plaintext = decryptor.update(ciphertext) +
            decryptor.finalize_with_tag(tag)

```

```

    except ValueError:
        print(f"Error: Decryption failed. Possible incorrect key or tampered
data.")
        return
    decrypted_file_path = "decrypted_" +
os.path.basename(enc_file_path.replace(".enc", ""))
    with open(decrypted_file_path, "wb") as f:
        f.write(plaintext)
    print(f"Decrypted file written to {decrypted_file_path}\n")

def add_user_to_group(username, user_cert_path):
    try:
        # Load existing group keys or initialize empty dict
        if os.path.exists("group_keys.json"):
            with open("group_keys.json", "r") as f:
                group_keys = json.load(f)
            else:
                group_keys = {}

        # Load the AES key for encryption (assumed to be previously generated and
stored)
        with open("aes_key_backup.json", "r") as f:
            aes_key = bytes.fromhex(json.load(f)["aes_key"])

        # Load the public key from the user's certificate
        public_key = load_public_key_from_cert(user_cert_path)
        if public_key is None:
            print(f"Error: Could not load public key from {user_cert_path}.")
            return

        # Encrypt the AES key for the new user using their public key
        encrypted_aes_key = encrypt_aes_key(aes_key, public_key)
        if encrypted_aes_key is None:
            print("Error: Failed to encrypt AES key for the new user.")
            return

        # Store the encrypted AES key in the group keys
        group_keys[username] = encrypted_aes_key

        # Save the updated group keys to file
        with open("group_keys.json", "w") as f:
            json.dump(group_keys, f, indent=4)

```

```

        # Ensure that the encrypted AES key is also updated in sample.txt.key
        with open("sample.txt.key", "w") as key_file:
            json.dump(group_keys, key_file, indent=4) # Append the new encrypted
key
            print(f"User {username} added to the group successfully.\n")
    except Exception as e:
        print(f"Error adding user {username} to the group: {e}\n")

def remove_user_from_group(username):
    try:
        # Ensure group_keys.json exists
        if not os.path.exists("group_keys.json"):
            print("Error: group_keys.json not found!")
            return

        # Load the current group keys
        with open("group_keys.json", "r") as f:
            group_keys = json.load(f)

        # Check if the user is in the group
        if username in group_keys:
            # Remove the user from the group
            del group_keys[username]
            print(f"User {username} removed from the group successfully.\n")
        else:
            print(f"User {username} not found in the group.\n")
            return

        # Save the updated group_keys.json file
        with open("group_keys.json", "w") as f:
            json.dump(group_keys, f, indent=4)

        # Rebuild the sample.txt.key file with the updated keys
        with open("sample.txt.key", "w") as key_file:
            for encrypted_key in group_keys.values():
                json.dump(group_keys, key_file, indent=4) # Re-write only the
remaining keys
    except Exception as e:
        print(f"Error removing user {username} from the group: {e}\n")

```

Test_script.py

```

import secure, os, json
print(" ----- TEST CASE 1: ALICE & BOB CAN USE THE FUNCTIONALITY  
INITIALLY ----- ")
print("Generating certificates for Alice and Bob...")
secure.generate_user_cert("Alice")
secure.generate_user_cert("Bob")

# Verify certificate generation
for user in ["Alice", "Bob"]:
    if not os.path.exists(f"{user}_public.pem"):
        print(f"Error: {user}_public.pem not found!")
    else:
        print(f"{user}'s certificate ({user}_public.pem) generated  
successfully.\n")

# Ensure private keys exist before proceeding
for user in ["Alice", "Bob"]:
    if not os.path.exists(f"{user}_private.pem"):
        print(f"Error: {user}_private.pem not found! Decryption may fail.\n")

secure.add_user_to_group("Alice", "Alice_public.pem")
secure.add_user_to_group("Bob", "Bob_public.pem")

# Define user keys (public keys for encryption)
user_keys = {"Alice": "Alice_public.pem", "Bob": "Bob_public.pem"}
print(f"User keys for encryption: {user_keys}")

print("Encrypting file sample.txt using the public keys of Alice and Bob...")
secure.encrypt_file("sample.txt", user_keys)

# Ensure the key file is created before uploading
if os.path.exists("sample.txt.key"):
    print("Encrypted key file (sample.txt.key) found, uploading...")
    secure.upload_to_cloud("sample.txt.key")
else:
    print("Error: sample.txt.key not found! Ensure encryption was successful.")

# Upload encrypted file
secure.upload_to_cloud("sample.txt.enc")

# Check if key file exists before proceeding with decryption
if os.path.exists("sample.txt.key"):
    with open("sample.txt.key", "r", encoding="utf-8") as f:

```

```

        encrypted_keys = json.load(f)

# Decrypt file for Alice using Alice's private key
if "Alice" in encrypted_keys:
    print("Decrypting the encrypted file for Alice using Alice's private key...")
    secure.decrypt_file("sample.txt.enc", "Alice_private.pem", "Alice")
else:
    print("Error: Encrypted key for Alice not found.")

# Decrypt file for Bob using Bob's private key
if "Bob" in encrypted_keys:
    print("Decrypting the encrypted file for Bob using Bob's private key...")
    secure.decrypt_file("sample.txt.enc", "Bob_private.pem", "Bob")
else:
    print("Error: Encrypted key for Bob not found.")

# Test Case 2: Adding Charlie to the user group after initial setup
print(" ----- TEST CASE 2: ADDING CHARLIE TO THE USER GROUP AFTER INITIAL SETUP ----- ")
print("Generating certificate for Charlie...")
secure.generate_user_cert("Charlie")
print("Adding Charlie to the user group...")
secure.add_user_to_group("Charlie", "Charlie_public.pem")

# Re-encrypt the file so Charlie gets the AES key
print("Re-encrypting file sample.txt with updated user list (Alice, Bob, Charlie)...")
user_keys = {"Alice": "Alice_public.pem", "Bob": "Bob_public.pem", "Charlie": "Charlie_public.pem"}
secure.encrypt_file("sample.txt", user_keys)

# Upload the updated key file
if os.path.exists("sample.txt.key"):
    print("Updated encrypted key file (sample.txt.key) found, uploading...")
    secure.upload_to_cloud("sample.txt.key")
else:
    print("Error: Updated sample.txt.key not found! Ensure re-encryption was successful.")

# Verify if Charlie can now decrypt the file
print("Decrypting the encrypted file for Charlie using Charlie's private key...")
secure.decrypt_file("sample.txt.enc", "Charlie_private.pem", "Charlie")

```

```
# Test Case 3: Removing Bob from the user group and ensuring he can no longer
decrypt
print(" ----- TEST CASE 3: REMOVING BOB FROM THE USER GROUP AND
ENSURING HE CAN NO LONGER DECRYPT ----- ")

print("Removing Bob from the user group...")
secure.remove_user_from_group("Bob")

# Verify that Bob cannot decrypt the file anymore
print("Attempting to decrypt the encrypted file for Bob using Bob's private
key...")
try:
    secure.decrypt_file("sample.txt.enc", "Bob_private.pem", "Bob")
except Exception as e:
    print(f"Bob correctly failed to decrypt the file. Error: {e}")
```